Incident Zero — Day 2 Plan

Oct 23, 2026 · @Tarun

Your Day 1 build is well ahead of where the plan expected it to be — seven tools, an evidence registry, a working loop, a practice harness, 25 pages of docs. But the last sweep scored 3/7 with **three confidently wrong answers**, which is the single worst outcome the brief names. Fixing that is worth more tomorrow than any new feature.

Where you actually stand

The 16:35 sweep is the only honest measure you have, and it says this:

	Result
Correct	3 of 7
Confidently wrong	3 (all at 0.85 high)
Escalated	0
Crashed	1 (Groq 429)

Every single correct answer came from the **error branch** — payment, cart, product-catalog. Every single wrong answer came from the **no-error branch** — the memory leak, the CPU spike, the traffic flood. That is not random. Your deepest-failing-span rule works, and there is no equivalent rule for faults that produce no errors.

The trap, in one line from the tool output. In all three failures, `get_service_health` returned this:

```
{"service": "accounting", "error_ratio": 0.0, "requests_per_second": 0.09,
 "p95_latency_ms": 15000.0, "status_hint": "SLOW", "ref": "metric-1"}
```

Accounting is a Kafka batch consumer. Fifteen seconds is its **normal**. But `_is_slower_than_baseline` ends with `return baseline is None or p95 >= 2 * baseline` — when the baseline is missing, it fails *open* and calls the service slow. Accounting runs at 0.09 requests per second, so its hourly median is sparse or absent, so it is labelled SLOW on every run. Your own tool handed the model a false lead, with a reference token attached, on every no-error incident.

Then the real answer was right there and got ignored. In `ad-high-cpu`, `get_container_health` returned `containers_with_problems: []` — and underneath it, `busiest_cpu` listing `ad` at **3.7 cores**, second only to the load generator. In `homepage-`

`flood`, `ad` at **10.4 cores**. The empty problems list reads as "all clear", so the model took the SLOW hint instead. In `recommendation-cache-leak` it went further: the tool actually flagged `recommendation` with `JUST_RESTARTED (uptime 42s)` — which is exactly what an OOM kill looks like — and the agent still answered `cart`.

Why this is the most expensive thing on the board. Checkpoint 4 is 30 points, scored automatically on unseen incidents, and the brief says the hidden set includes infrastructure faults — the no-error kind, the kind you currently get wrong three times out of three. At 43% you are looking at roughly 13 of 30. At 6/7 you are looking at roughly 25. That is twelve points from a handful of lines, which is more than the entire judges' demo allocation.

One piece of good news: your current `tools.py` already has `cpu_baseline`, `CPU_SPIKE` detection and a `cpu_baseline` field in `busiest_cpu` — none of which appear in the 16:35 transcripts. You improved the tools after that sweep and never re-ran it. **You do not actually know your current score.** That is the first thing to fix tomorrow, before writing anything new.

The real clock

Day 2 looks like a nine-hour day. It is not. Read the milestone row again:

- 09:00–09:30 alignment (no coding)
- 09:30–12:00 build the action layer
- 12:00 **Checkpoint 3: Hands** — one full run, alert to verified recovery, with the action log
- 12:00–13:30 harden and document — **"no new features"**, explicitly
- 13:30–14:00 final verification and submission
- 14:00 **code freeze, hard cutoff** — later commits ignored

You have two and a half hours of new code. Everything after 12:00 is bug-fixing, a clean run from scratch, and paperwork. Plan as if the day ends at 12:00, because for building purposes it does.

The deliverables slide contradicts this and says the freeze is 16:30. Two slides, two numbers. **Assume 14:00 and ask at 09:00.** If it turns out to be 16:30 you have a bonus afternoon; if you assume 16:30 and it is 14:00 you lose everything unfinished.

What 2.5 hours forces:

Do	Don't
One action (`restart_service`) working end to end, perfectly	All four actions half-working

Do	Don't
The baseline fix — it protects 30 points	A chat UI (explicitly optional)
Dry-run, approval, verify, rollback, kill switch	Kubernetes scaling you cannot test on Compose
Re-run the sweep to measure what you have	Refactoring code that already works

The brief's own minimum bar is *"a full loop on at least **one** incident"*. One incident, done properly, with evidence and a verified recovery, scores the full 20 at Checkpoint 3. Four incidents half-wired scores less and risks the clean run.

The fix worth more than every feature

Your error branch has a causal rule computed in Python: *the failing component is the deepest erroring span whose own children did not error*. It is deterministic, it is right every time, and it is why you score 3/3 on application faults.

Your no-error branch has no such rule. It hands the model a list of services with absolute numbers and lets it pick. That is guessing dressed as reasoning, and it is why you score 0/3 on infrastructure faults.

The twin rule is: **for faults that produce no errors, the root cause is the service whose own behaviour deviates most from its own normal.** Not the slowest service — the most *changed* service. Accounting at 15 seconds is not changed. Ad at 10 cores when it normally uses 0.3 is changed by a factor of thirty.

Three concrete pieces, in order of value:

1. Make the missing baseline fail closed, not open. One line, thirty seconds, and it alone removes the false lead that caused all three failures:

```
def _is_slower_than_baseline(row):
    p95, baseline = row["p95_latency_ms"], row["p95_baseline_ms"]
    if not p95 or p95 < 500:
        return False
    if baseline is None:
        return False          # no baseline = we do not know what normal is,
so this is NOT evidence
    return p95 >= 2 * baseline
```

A service you have no baseline for cannot be called anomalous. Say so in the `status_hint` (`"unknown (no baseline)"`) so the model sees the absence rather than a false positive.

2. Rank every service by deviation, and put that ranking first. Add one computed field to each row in `get_service_health` and sort on it:

```
def _deviation_score(row):
    # how many times worse than its own normal is this service right now?
    # None baseline contributes nothing: unknown is not the same as bad.
    error_delta = row["error_ratio"] - (row["error_ratio_baseline"] or
row["error_ratio"])
    latency_ratio = 1.0
    if row["p95_latency_ms"] and row["p95_baseline_ms"]:
        latency_ratio = row["p95_latency_ms"] / max(row["p95_baseline_ms"],
1.0)
    return round(max(error_delta * 20.0, latency_ratio - 1.0), 3)
```

Return the list sorted by `deviation_score` descending, and change the tool's `note` to say *"services are ranked by how far they have moved from their own baseline; a high absolute number with deviation_score near 0 is normal for that service."* The model reads the first row first. Make the first row the right one.

3. Give `get_container_health` the same treatment, and stop burying the answer. You already compute `cpu_baseline`. Three changes:

- Promote anything above about 3× its CPU baseline, or with memory climbing past ~70% of its limit, into `containers_with_problems` — not into the `busiest_cpu` footnote.
- Never return `containers_with_problems: []` alongside a 10-core outlier. If the list would be empty but a container is more than 3× baseline on anything, it goes in the list.
- Add a `memory_trend` field: memory now versus memory at the start of the window. A cache leak shows as a rising line long before it hits the limit, and this is the only signal that catches `recommendationCacheFailure` before the OOM kill.

The honesty dividend. Once deviation is computed, your confidence rule gets sharper for free: *high confidence requires the named service to be the top-ranked deviation **and** have two evidence kinds*. If the top deviation and the named service disagree, that is exactly the situation where an escalation beats a guess — and the brief pays you for escalating.

Five things that make this outstanding

Most teams will arrive at 12:00 with an agent that restarts a container after someone types `y`. That satisfies the brief. These five are what separate you, ordered by value per minute spent — and three of them cost almost nothing because the work is already done.

1. A measured calibration curve. Cost: 20 minutes. Nobody else will have one.

Stretch goal 3 asks you to show that stated confidence matches how often the agent is right. You already produce every input for this in `reports/sweep-*/scoring-table.md`. Write ten lines that aggregate across sweeps:

```
Stated confidence   Runs   Correct   Actual accuracy
high (>=0.75)        6       5          83%
medium (0.5-0.74)    4       3          75%
low (<0.5)           3       0          --  (all escalated, none claimed)
```

This is the single most persuasive artefact you can put in front of a judge scoring *testing and honesty*. It converts "we think it is calibrated" into a number. Put it in `evaluation-note.md` and say the words out loud in the demo.

2. A reviewer agent that tries to disprove the diagnosis. Cost: 30 minutes. Directly fixes your worst failure.

Stretch goal 2, and it is aimed squarely at your three confidently-wrong answers. After `submit_diagnosis` validates and before the human sees anything, make one more LLM call with the evidence and the named service, prompted to argue the opposite: *"Here is a proposed root cause and its evidence. Name the strongest reason this is wrong, and name a service the evidence fits better. If you cannot, say CONFIRMED."*

If the reviewer dissents, drop confidence one band and attach its objection to the report. An agent that publicly argues with itself and shows the transcript is far more convincing than one that is merely right.

3. Rollback on failed recovery. Cost: 15 minutes. Most teams will skip it.

The brief says plainly: *"If it did not, roll back and escalate."* Most teams will act, check health, print "recovered", and stop. Capture the pre-action state, act, re-check after 90 seconds, and if the metric has not improved, undo the action and escalate with both the action log and the failed verification. This is one `if` and an inverse action map, and it is the difference between "we automated a fix" and "we automated a *safe* fix".

4. Writing flags is allowed; reading them is disqualification. Say so explicitly. Cost: 0.

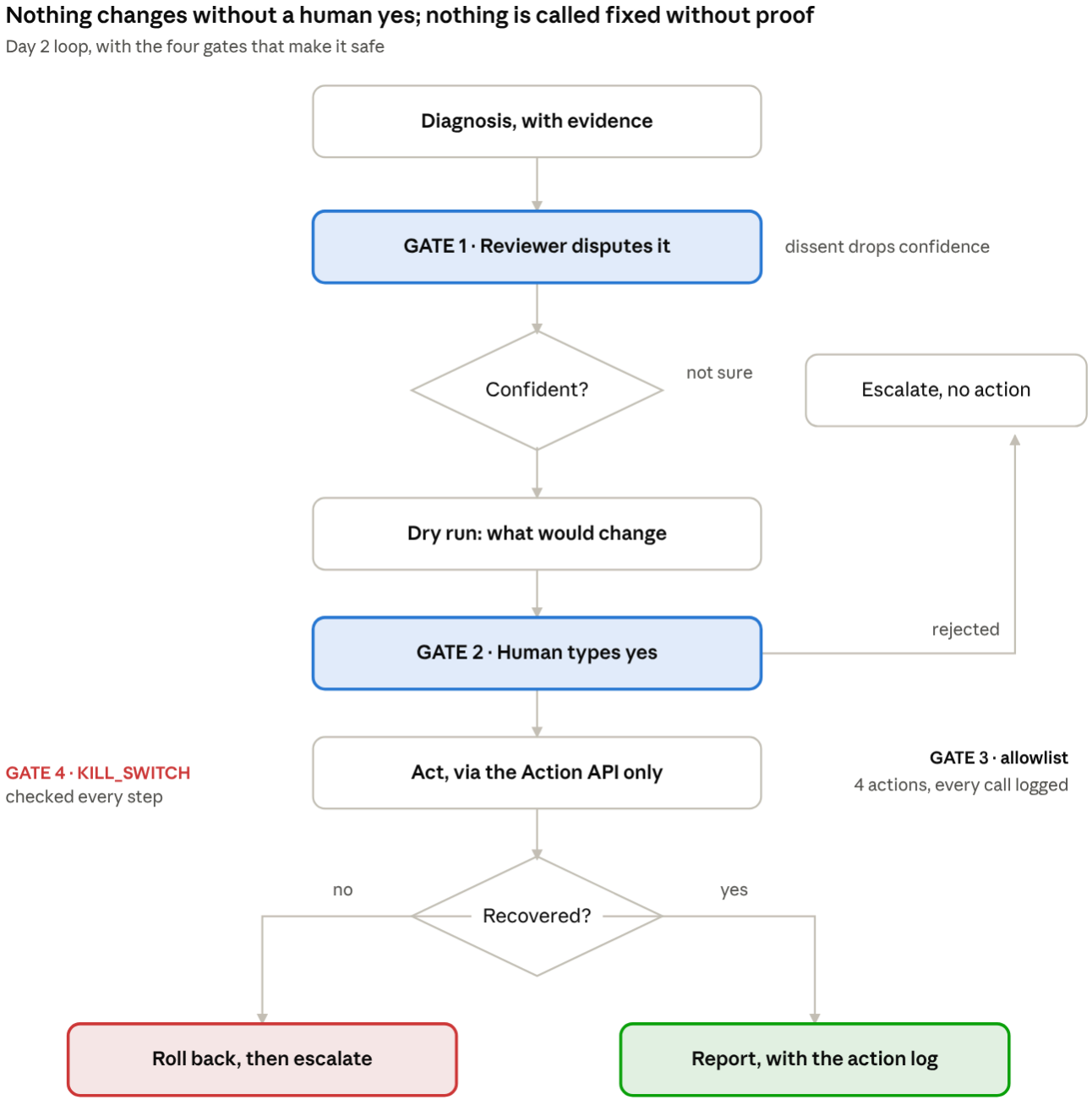
Rule 2 bans reading feature-flag settings to diagnose. The Action API's allowlist includes *toggling a feature flag* as a remedy. Those are not in conflict, but a judge will be watching to see whether you understand the line. You already redact `feature_flag.*` from telemetry in the tool layer — which is stricter than required and a genuinely good decision.

Make the asymmetry a design statement in the README and the demo: **the diagnosis path cannot read flags, the action path can only write them, and no code reads a flag to decide anything.** One sentence, and it tells the judges you read the rules more carefully than the teams who merely avoided the flag UI.

5. Live proof that invented evidence cannot get through. Cost: 10 minutes.

Your evidence registry already rejects references no tool returned. Nobody will believe it unless you show it. Add a hidden `--demo-hallucination` flag that injects one fake reference into the model's submission, so the guard fires on camera and the model is forced to resubmit with real evidence. Thirty seconds of demo that no other team can match, and it makes the *"invented evidence is penalised"* line on the brief work in your favour instead of being a risk you survived.

The loop you are building



Day 2 act-and-verify loop · 4 safety gates

Four gates, and each one maps to a scored criterion. The reviewer is stretch goal 2, the approval gate and allowlist are Checkpoint 3's core, the kill switch is a stated minimum, and the rollback branch is the sentence most teams will skip.

Hour by hour

09:00–09:15 — alignment, and one person starts the stack immediately. The demo takes three minutes to come up and metrics lag two more. Someone runs `dc up -d --remove-orphans` while the briefing happens. Ask the four questions in the risks section.

09:15–09:30 — re-run the sweep before touching anything. You improved `tools.py` after the 16:35 sweep and never measured the result. Start `python run_all.py` now, in the background, and let it run while you work. Whatever it says is your true starting score and your before-number for the evaluation note.

09:30–10:00 — the baseline fix. One person, thirty minutes, the three changes in the section above. This is the highest-value half hour of the day. Do not let it expand.

10:00–11:15 — the action layer. Two people in parallel, on separate files so they do not collide:

Who	Builds
A	`actions.py` — the allowlist, dry-run, execute, inverse map, append-only `action_log.jsonl`
B	`verify.py` — pre-action snapshot, post-action re-check after 90s, the recovered/not-recovered verdict
C	Wires both into `agent.py`: the approval prompt, the kill-switch check, the rollback branch

11:15–11:45 — the full loop, end to end, three times. Inject `payment-failure`, wait 120 seconds, run, approve, watch it act, watch it verify, read the report. Then do it again having rejected the action. Then do it again with `touch KILL_SWITCH` mid-run. All three paths must work, because the mentor will ask for at least one of them.

11:45–12:00 — freeze for the checkpoint and rehearse. Terminal open, command typed, not entered.

12:00 — Checkpoint 3. Show one full run: alert to verified recovery, with the action log.

12:00–13:00 — harden. No new features. Re-run the sweep a second time with the baseline fix in, so you have before-and-after numbers. Build the calibration table from both sweeps. Clean run from a `dc down && dc up -d`. Fix only what is broken.

13:00–13:30 — the five deliverables. README with setup, how to run, how the approval gate and kill switch work, and known limitations. The one-page architecture overview (a photo of a whiteboard is explicitly allowed). The incident reports in schema plus readable form. The evaluation note with both sweeps and the calibration table. The team form naming who built what and declaring every AI model used, Groq and any coding assistant included.

13:30–14:00 — final verification. Validate every report against the schema. `git log --format='%an' | sort -u` to confirm all five names. Push. Tag.

14:00 — code freeze. Hands off.

14:00–15:30 — Checkpoint 4, hidden test. The organisers run unseen incidents on your setup. Your job is to keep the stack healthy and not touch the code. Have the demo reset to a clean state and running.

15:30–18:45 — evaluation, then finalist demos. If you are shortlisted: five minutes live, three minutes of questions, plus a one-to-one. Everyone must be able to explain the whole pipeline, not only their own file.

The five-minute demo

Five minutes live, three minutes of questions, no video. Rehearse it twice at 11:45 and once more before you are called. Inject the fault **before** you walk up, so the two-minute metric lag has already passed.

0:00–0:30 — the problem, in one sentence. "When payment breaks, five services report errors and the alert names the frontend. Only one of them is broken. Here is an agent that finds which one, and proves it."

0:30–1:30 — the money shot. Run it live. While it works, say the one thing that matters: *the root cause is the deepest failing span whose own children did not fail, and we compute that in Python, not in the prompt.* Then show the output naming `payment` while `checkout`, `frontend`, `frontend-proxy` and `load-generator` all carried errors.

1:30–2:00 — the evidence guard, live. Run with `--demo-hallucination`. The registry rejects the invented reference on screen and the model resubmits with a real one. Say: "invented evidence cannot reach our output, because it is checked in code against what the tools actually returned."

2:00–3:00 — the safe fix. Show the dry run: here is what would change, and nothing has changed yet. Type the approval. Watch it act, then verify recovery against the metric it started from. Show `action_log.jsonl`.

3:00–3:30 — the kill switch. `touch KILL_SWITCH` mid-run, show it stop. Three seconds of work, and it is a stated minimum.

3:30–4:15 — honesty, which is where you win. Put the calibration table on screen: when we said high confidence, we were right N times out of M. Then say what does not work, out loud, before anyone asks. Rule 7 is explicit that stating a limitation costs nothing and hiding a failure loses points — so name the Kafka incidents, the provisional schema, the stand-in faultctl.

4:15–5:00 — the design decision nobody else made. The flag asymmetry: the diagnosis path physically cannot read feature flags, the action path can only write them, and `feature_flag.*` is redacted in the tool layer before the model ever sees it. That is one sentence that tells a judge you read rule 2 properly.

Questions you will get, and the short answers.

- *What if the LLM is wrong?* The reviewer pass argues against every diagnosis, two evidence kinds are required for high confidence, and the escalation path is a correct outcome rather than a failure.
- *Why not a framework?* A plain tool-calling loop is about eighty lines, every member can explain it, and we can show you the exact line where any decision is made.
- *How do you know it works?* Two measured sweeps across the practice set, before and after the baseline fix, with the numbers in `evaluation-note.md`.
- *What breaks it?* Answer honestly from your known-limitations list. Do not improvise a defence.

Risks, worst first

1. The Groq 429 will cost you the hidden test. Your `payment-unreachable` run died on a rate limit, and `llm.py` already backs off through 5/15/30/60/90 seconds. Backoff does not help against a free-tier **daily** cap, which is what a seven-incident sweep plus a morning of testing will hit. During Checkpoint 4 the organisers run unseen incidents on your setup — if the key is exhausted at 14:30, you score zero on thirty points, having done nothing wrong.

Fix it in the first twenty minutes: a second key from a different account in `.env.local` as `API_KEY_FALLBACK`, and in `llm.py`, on a `RateLimitError` that survives two retries, swap client and model and carry on. Log which model answered, into the transcript, because

the team form must declare every model used. Ask the organisers at 09:00 whether they are providing a key — the brief lists LLM access with a spend limit in the starter kit, and theirs will have a higher ceiling than a free Groq tier.

2. The schema swap. Yours is provisional and you flagged it honestly. When the organisers' arrives, replace `schema.json` verbatim and change only the field names in `build_diagnosis`. Keep the evidence registry and the calibration logic exactly as they are — they are yours and they are the good part. Budget twenty minutes and do it before 11:00, not at 13:45.

3. `scale_service` does not exist on Docker Compose. The Action API allowlist names it as Kubernetes-only. Do not fake it. Return a structured refusal — `{"ok": false, "reason": "scale_service requires Kubernetes; this deployment is Docker Compose"}` — and say so in the README. An honest unsupported action is worth more than a broken one, and rule 7 pays for exactly this.

4. Your faultctl is a stand-in. If the organisers' differs, your `run_all.py` sweep breaks at the worst moment. Keep the injection call behind one function so swapping it is a single edit, and do not let the sweep harness become load-bearing for the demo.

5. Two people editing `agent.py` at once. You are five on one repo with a 14:00 freeze. `git pull --rebase` before every push, and keep `actions.py` and `verify.py` as new files so the merge surface stays small.

Ask these at 09:00

1. **Code freeze: 14:00 or 16:30?** The milestones slide and the deliverables slide disagree. Get a number.
2. **Are `feature_flag.*` attributes in telemetry allowed as evidence, or does using them count as reading flag settings?** You currently redact them, which is the safe side. If allowed, unredacting is a one-line win; if not, you are already correct.
3. **Does the hidden-test harness call our agent, or must it self-detect?** If it calls you, it needs a clean entry point — you already have `diagnose(alert_text) -> dict` and a CLI, so confirm which shape they want.
4. **What is the deploy-history source?** Yours is a `docker inspect` stand-in.
5. **Is an LLM key provided, and what is the spend limit?** Related to risk 1.

Tonight

Do not commit code tonight. Rule 1 says all code is written on the day and commits before then are not allowed. A judge scrolling `git log` and finding commits timestamped 23:40 between Day 1 and Day 2 is a conversation you cannot win, and the two hours you would

save are not worth a disqualification question. The Build runbook tab holds the code ready to type at 09:30; that is the whole point of it.

Four things that are not code and are worth doing now:

1. **Get a second Groq key** from a different account, or sign up for a fallback provider. Put it in `.env.local`, which is gitignored. This is account setup, not code.
2. **Send the team the Day 1 state.** Four people have not read `SETUP_NOTES.md`, and two of them are about to write the action layer against a Jaeger API whose path is not what the docs say. Ten minutes of reading tonight saves an hour tomorrow.
3. **Decide who owns what** — the role table in the hour-by-hour section. Agreeing at 09:30 costs thirty minutes of the two and a half you have.
4. **Read the brief's deliverables list once more.** Five items, and the team form is the one teams forget until 13:55.

Sleep. You have been debugging Docker since 14:00 and the day that matters starts at 09:00 with two and a half hours of build time. Tired decisions at 11:00 tomorrow will cost more than anything you could add tonight.